Official 2D c-tag Scale Factors

Working points, calibration, and what they cost the limit

Chirayu Gupta — VUB

2026-07-31

BTV SFbc-2D scheme (AN-25-222) · PNet AK4 · H+c → WW, 2022postEE

What this deck answers

1. **What** the 2D scheme is, and why it replaces single-cut working points.
2. **How** our stored variables map onto the official axes — and why no reprocessing was needed.
3. **Which** categories are actually populated in our analysis (fewer than you'd think).
4. **What** the official per-category SFs are, and how wide their uncertainty is.
5. **Closure: the limit with and without the SF, everything else held fixed.**

Every number here is measured on the `hww_combine_fixed` 2022postEE workflow.

The with/without comparison is a controlled A/B — the no-SF variant is built from the pre-SF backup parquets through the *same* builder, on the *same* day, with only the `CMS_ctag2d_2022` nuisance removed.

Part 1 — The 2D scheme

From working points to a plane

The BTV **SFbc-2D** calibration (AN-25-222) replaces a single-cut working point with a **2D plane** partitioned into **11 categories** — `L0, C0–C4, B0–B4` — each carrying its own scale factor, with **frozen** bin edges.

axis	definition
BvC (y)	$1 - \mathrm{CvB}$
HFvLF (x)	$P_b + P_c = 1 - P_L$

Frozen edges (official, 2026-06-29)

```
HFvLF (x): [0.000, 0.250, 0.452, 0.808, 1.000]
BvC   (y): [0.000, 0.006, 0.017, 0.055,
            0.761, 0.944, 0.985, 0.995, 1.000]
```

Use the official frozen edges — inventing our own would re-randomise the bins each run and break the published SF lookup entirely.

Integer ids used everywhere:

`L0=0, C0=1, C1=2, C2=3, C3=4, C4=5, B0=6, B1=7, B2=8, B3=9, B4=10`.

Key result: the axes need only CvsL and CvsB

The scheme as sketched needs the raw b-score, which **our parquets never stored**.
It turns out not to matter.

CMS PNet AK4 discriminants share one 3-simplex $(b,\ c,\ L \equiv uds + g)$:

$$\mathrm{CvsL} = \frac{P_c}{P_c + P_L}, \qquad \mathrm{CvsB} = \frac{P_c}{P_c + P_b}, \qquad P_b + P_c + P_L = 1$$

Three equations, three unknowns → **unique** solution:

$$P_b = \frac{\mathrm{CvL}\,(\mathrm{CvB} - 1)}{\mathrm{CvB} \cdot \mathrm{CvL} - \mathrm{CvB} - \mathrm{CvL}}, \qquad \mathrm{HFvLF} = \frac{\mathrm{CvL}}{\mathrm{CvL} + \mathrm{CvB}(1 - \mathrm{CvL})}, \qquad \mathrm{BvC} = 1 - \mathrm{CvB}$$

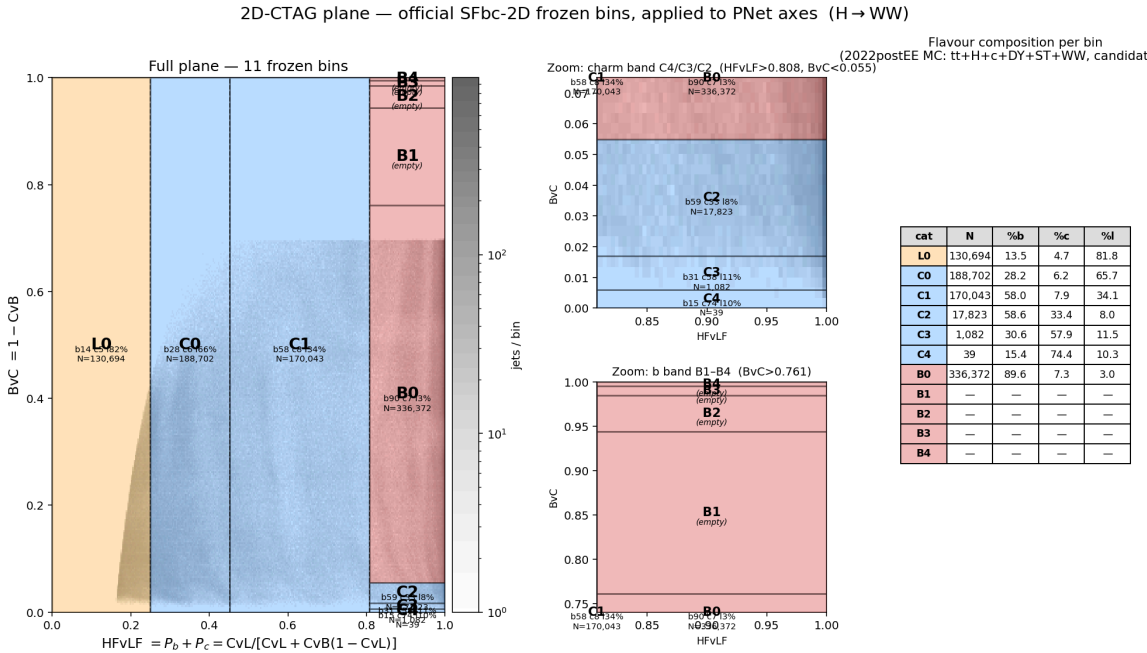
Consequence: no NanoAOD reprocessing was ever needed — the categories are computed directly from the two stored columns.

Verified three independent ways

check	result
Symbolic (sympy)	unique closed form exists
2M random valid probability triplets	max $\left\|B_{\mathrm{rec}} - B\right|$ = **4×10⁻¹⁶** (machine precision)
30k real jets, `Jet_btagPNetB` from a Run3Summer22 **NanoAODv12** file	median **2.8×10⁻⁵**, 95pct 1.4×10⁻⁴

The ~10^{-5} residual on real data is NanoAOD **float-storage rounding**, not a modelling gap — a gluon-class mismatch would show O(0.1–1) errors, not O(10^{-5}). So "L" in CvsL lumping $uds + g$ is consistent, and the recovery is exact up to storage precision.

The plane, plotted



cat	N	%b	%c	%l
L0	130,694	13.5	4.7	81.8
C0	188,702	28.2	6.2	65.7
C1	170,043	58.0	7.9	34.1
C2	17,823	58.6	33.4	8.0
C3	1,082	30.6	57.9	11.5
C4	39	15.4	74.4	10.3
B0	336,372	89.6	7.3	3.0
B1	—	—	—	—
B2	—	—	—	—
B3	—	—	—	—
B4	—	—	—	—

Density = 2022postEE MC (tt + H+c + DY + Single Top + WW), **candidate c-jet**,

845k jets. Insets zoom the two thin bands, which are invisible at full scale.

⚠️ Only 7 of 11 categories are populated

cat	N	%b	%c	%l
L0	130,694	13.5	4.7	**81.8**
C0	188,702	28.2	6.2	65.7
C1	170,043	58.0	7.9	34.1
C2	17,823	58.6	33.4	8.0
C3	1,082	30.6	**57.9**	11.5
C4	39	15.4	**74.4**	10.3
B0	336,372	**89.6**	7.3	3.0
B1–B4	**0**	—	—	—

Why: the c-jet *candidate* is already charm-selected, so it sits at high `CvB`, i.e.
low BvC. The B1–B4 band ($\mathrm{BvC} > 0.761$) is **cut away by construction**.

- Charm purity rises correctly C2→C3→C4 (33% → 58% → 74%), so the *ordering* is physical —
  but the high-purity charm bins hold very few jets.
- On the **leading jet** (not charm-selected) the full ladder does populate, and b-purity
  rises monotonically B0→B4: 24 → 75 → 93 → 98 → 100%.

The scheme is sound; our candidate-c-jet selection truncates it.

Part 2 — The scale factors

Source and signature

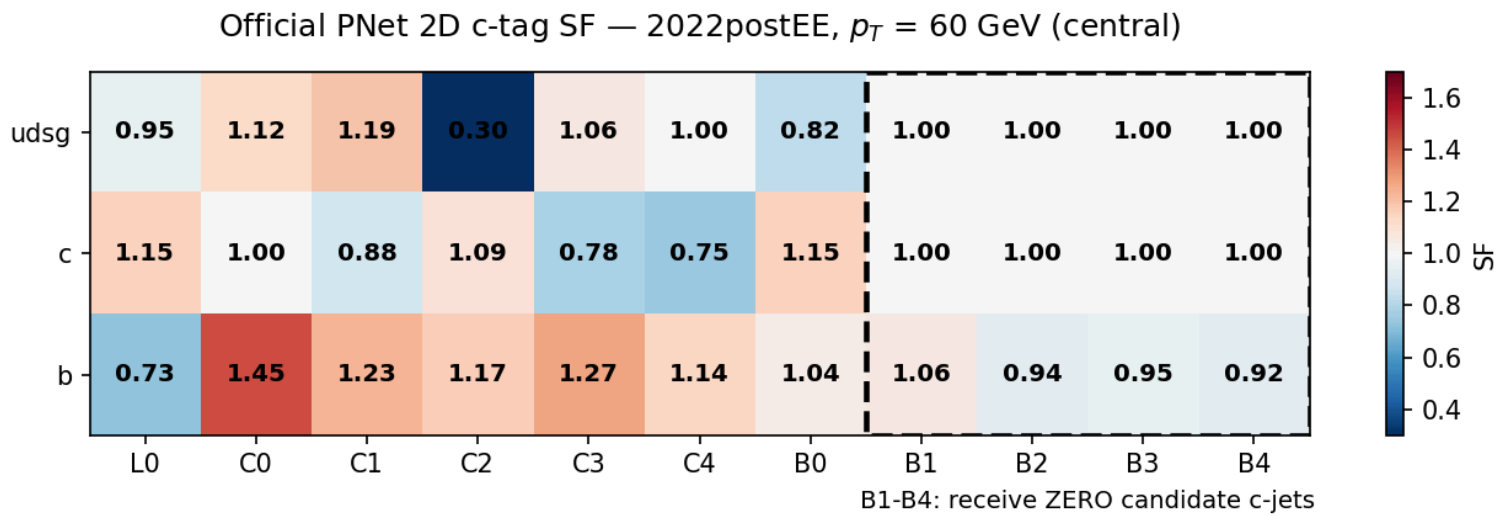
Official **per-category** 2D flavour-tagging SFs for PNet AK4 — the H→γγ / `cmshgg` "2D_HF_Tagging" ingredients, correctionlib v2.

campaign	file
2022 preEE	`.../ingredients/2022/2D_HF_Tagging/flavTaggingSF_2022preEE.json.gz`
2022 postEE	`.../ingredients/2022/2D_HF_Tagging/flavTaggingSF_2022postEE.json.gz`
2023 preBPix	`.../ingredients/2023/2D_HF_Tagging/flavTaggingSF_2023preBPix.json.gz`
2023 postBPix	`.../ingredients/2023/2D_HF_Tagging/flavTaggingSF_2023postBPix.json.gz`

Correction: `ParticleNetAK4_pseudocontinuous` · **Signature:** `evaluate(systematic, flavor, wp, abseta, pt)`

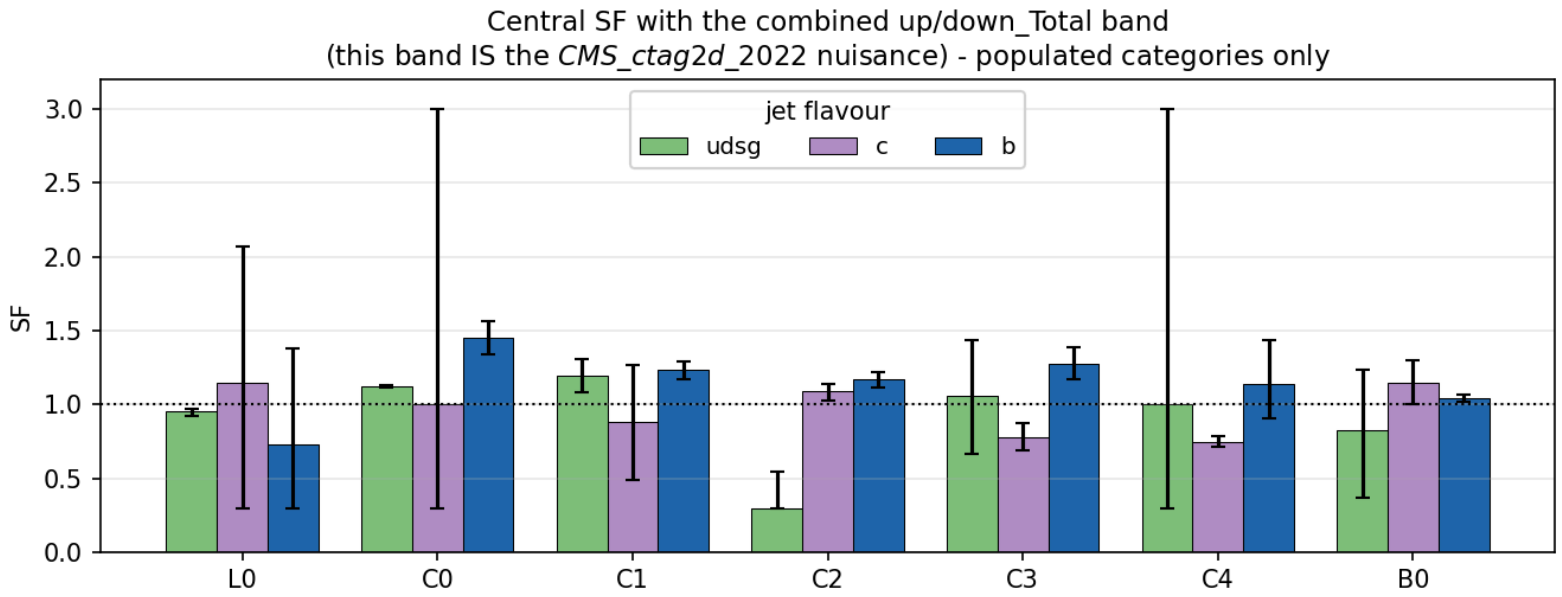
- `systematic`: `central`; `up_Total` / `down_Total` (combined — **use this for one nuisance**), `up_Stat` / `down_Stat`, plus a large per-source decomposition. **There is no bare `up` / `down`.**
- `flavor`: `0`=udsg, `4`=c, `5`=b — the jet's **hadron flavour** (`cjet_cand_flavour`).
- `wp`: the 2D category id — **`L0=0, C0..C4 = 40..44, B0..B4 = 50..54`**
  (note: **not** our stored 0..10 — must be remapped).
- `abseta`: **inclusive** (single bin, value irrelevant), `pt`: binned `[20,35,50,70,90,120]`.

The central SF matrix



Evaluated at p_T = 60 GeV. Corrections are **O(10–30%)** in the populated categories — not small. The b-row is well determined everywhere (b-jets are abundant); the udsg C2 value of 0.30 is a floor on a nearly-empty light-flavour cell.

The uncertainty band — this *is* the nuisance



`up_Total` / `down_Total`, populated categories only. Where a category is well populated for that flavour the band is a few percent (b in C1/C2/B0). Where it is **starved**, the band hits the ±[0.3, 3.0] guard rails — c in C0, udsg in C3/C4.

A single `Total` nuisance inherits the *widest* of these.

How it enters the fit

The SF is a genuine correction on the candidate c-jet, so it multiplies the **nominal** weight, with one combined shape+norm nuisance `CMS_ctag2d_<year>` carrying `up_Total` / `down_Total`.

Applier `b-hive/scripts/apply_ctag2d_sf.py` adds three columns per MC `mva/<sample>.parquet` (data untouched; idempotent, `.bak_pre_ctag2dsf` backup):

- `weight_nominal` ← `weight_nominal × SF_central` — and **every existing `weight_*` variation is scaled by the same central SF**, so the correction is everywhere.
- `weight_CMS_ctag2d_<year>Up` = `weight_nominal_corrected × SF_upTotal / SF_central`
- `weight_CMS_ctag2d_<year>Down` = `weight_nominal_corrected × SF_downTotal / SF_central`

Category per row is recomputed from `cvsl_pnet` / `cvsb_pnet`; flavour from `cjet_cand_flavour`; rows with no candidate c-jet (`cjet_cand_pt` NaN) get SF = 1.

Object-shift consistency: the JES/JER/lepton templates read `weight_nominal` from their **own** shift dirs. The SF is applied there too — recomputed from each dir's *shifted* scores/pt — so each shift is measured against the same SF-corrected baseline. Otherwise every object-shift nuisance would pick up a spurious ~6% offset.
All 12 shift dirs corrected.

Part 3 — Closure: with vs without the SF

How the A/B was built

To make this a **controlled** comparison rather than a comparison against an old number, both datacards were rebuilt from scratch, the same day, through the same builder:

	no-SF variant	with-SF variant
workflow	`hww_combine_nosf`	`hww_combine_sfchk`
`mva/` parquets	symlinks → **585** `.bak_pre_ctag2dsf` files	the live SF-corrected files
yaml	`hww_combine_fixed.yaml` **minus one line**	`hww_combine_fixed.yaml` verbatim
nuisance count	**29**	**30**
negrw	✅ identical treatment	✅ identical treatment

The two cards differ by exactly one nuisance row and the central-SF weight rescale. Nothing was overwritten — the baseline `hww_combine_fixed` tree is untouched, and the no-SF tree is symlinks only (no data copied).

Both are built **after** the negative-weight reweighting, so the negrw renorm factors appear identically in both build logs.

⚠️ A normalization trap found while doing this

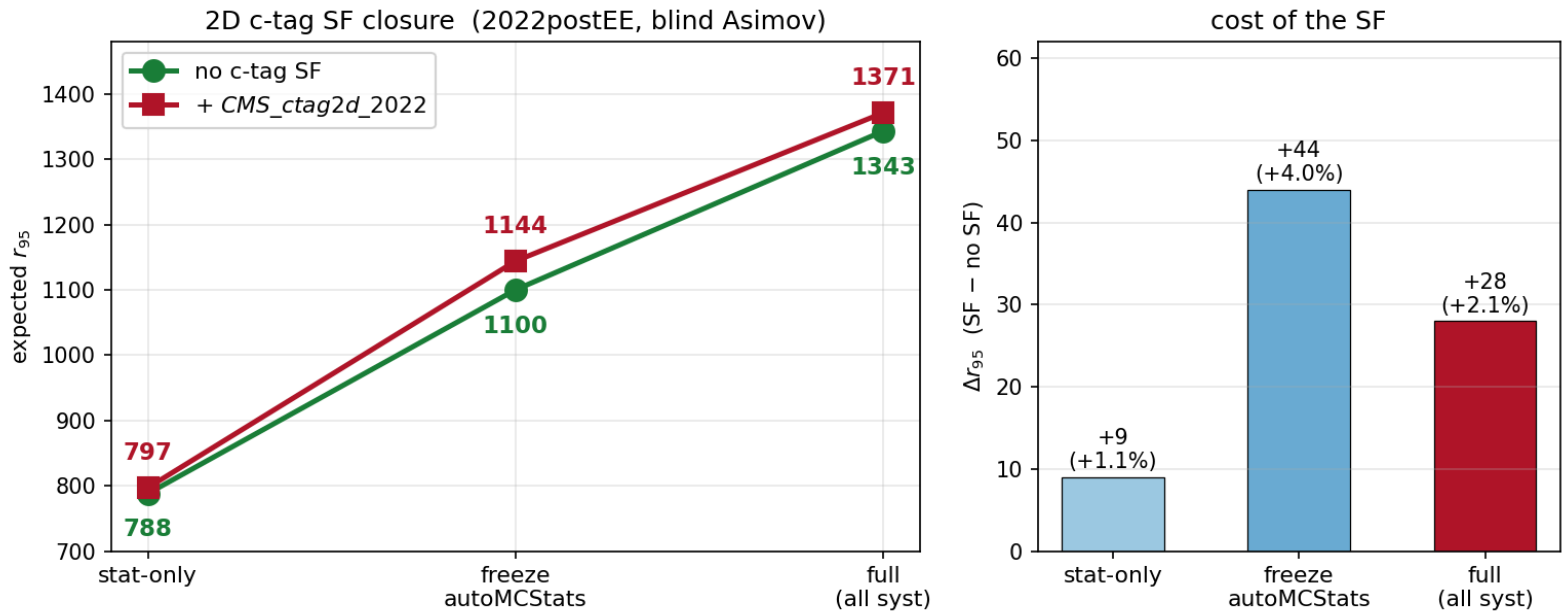
The builder has **two** `read_scale` implementations. Only the **sidecar** one (`analysis/filesets/sumw_<year>.json`) is correct; the parquet-metadata one **undercounts**, because those parquets predate the `dump_chunk_sumw` processor fix.

sample	sidecar `sumw`	parquet-md `sumw`	ratio
WtoLNu_2Jets	4.94e+13	8.52e+12	**5.80×**
TbarQto2Q	1.02e+09	1.41e+07	**72.6×**
WWZ / WZZ / ZZZ	~1e6	**0.0**	no metadata at all
DY, tt, Higgs, ST	—	—	1.000 ✅

`sumw` **is the denominator** of $\mathrm{lumi} \times \sigma / \mathrm{sumw}$, so undercounting **inflates** the yield — and it lands on `WtoLNu`, squarely on V+jets.

Dangerous because it is common-mode: two variants built the same wrong way still agree with *each other* to ~1%, so the A/B looks internally consistent while both numbers are wrong. **Check after any rebuild:** SR V+jets ≈ **735**, all-channel V+jets ≈ **5.8k** (2022postEE). If V+jets is ~14k, you are on the parquet-metadata path.

The closure result



Measured **2026-07-31** from freshly rebuilt inputs, both arms through the same builder on the same day. **The with-SF arm reproduces the reference datacard bin-for-bin** (SR V+jets 745.9, all-channel 6163.1, SR total 9141.5 — identical to `v11_hplusc_v4.root`), so the A/B is verified end-to-end, not just internally consistent.

Reading the closure

variant	full (all syst)	stat-only	freeze autoMCStats
baseline, **no SF**	**1343**	788	1100
+ `CMS_ctag2d_2022`	**1371**	797	1144
Δ	**+28 (+2.1%)**	+9 (+1.1%)	+44 (+4.0%)

The SF weakens the limit by ~2% — and that is the correct sign.

A real c-tag scale factor *should* cost something: it introduces a genuine
uncertainty that was previously being ignored, not assumed away.

Decomposition

- **stat-only +1.1%** — pure yield rescaling. Mean central SF ≈ 1.06, so SR $S/\sqrt{B}$
  shifts slightly. Not an uncertainty effect at all.
- **full +2.1%** — the extra degradation is the one new nuisance's wide
  (+44%/−16%) band doing its job.

The stat-only → full gap is still autoMCStats-dominated (freeze → 1144),
i.e. low-stat SR templates remain the driver, **not** the SF.

The matched combination: 2D-cat MVA + SF

The SFs calibrate the **2D-category** tagging, so they physically belong with the 2D-cat MVA scores, not the baseline continuous scores. Built as a separate workflow `hww_combine_2dcat`.

variant	full	stat-only	freeze autoMCStats
baseline (no SF)	1343	788	1100
baseline scores + SF	1371	797	1144
2D-cat scores + SF	**1422**	**749**	1168

Stat-only improves −5% (749 vs 788).
The 2D-cat MVA is the sharper discriminant: signal $\langle P_{\mathrm{hplusc}} \rangle$ rises to 0.514 (from 0.377) and **91.1%** of signal lands in the SR (from 70.8%).
Consistent with its +0.004 AUC.

Full limit is worse +6% (1422).
The 2D-cat argmax also pulls **2.3× more tt** into the SR (tt→SR 18.9% vs 8.3%), so SR yields roughly double. tt is the dominant background, so the enlarged SR is more systematics-exposed.

Verdict: an honest separation-vs-systematics trade, not a bug. Levers to recover it: tighten the SR, split ggH out of higgsbkg, or narrow the SF nuisance using a less conservative decomposition than `Total`.

Does the 2D categorisation cost MVA performance?

Retrain of v11 (6-class) with the two continuous PNet scores **removed** and the 11 one-hot categories **added**:

Discriminant	2D-cats	Baseline	Δ
hplusc_vs_all	**0.9322**	0.9284	**+0.0038**
hplusc_vs_higgsbkg	0.8302	0.8506	**−0.0204**
hplusc_vs_tt	0.9438	0.9375	+0.0063
hplusc_vs_st	0.9334	0.9287	+0.0047
hplusc_vs_diboson	0.8855	0.8840	+0.0015
hplusc_vs_vjets	0.9425	0.9348	+0.0077

≈ equivalent, marginally better overall, 4 of 5 background ROCs improve.

The one regression is vs **higgs-background** — separating H+c from other Higgs modes needs the fine charm-tag gradient a coarse 11-bin (effectively ~6-bin) scheme discards.

…but the charm tag is a small part of the MVA

Top features (2D-cats model, 26 inputs)

#	feature	rel%
1	dilepton_mass	25.4%
2	mtl1	17.6%
3	met_pt	11.9%
4	mtl2	10.6%
5	dilepton_pt	8.2%
6	lepton1_pt	8.1%
…		
11	**ctag2d_B0**	1.2%
12	**ctag2d_L0**	0.6%
26	**ctag2d_C4**	**0.00%**

model	charm-tag total	share
2D-cats (11 one-hot)	0.03064	**2.8%**
baseline (2 scores)	0.02340	**1.9%**

1. **Kinematics dominate both models** — `dilepton_mass` + `mtl1` alone are ~43%.
   This is the headline caveat on *any* c-tag change.
2. The 2D block carries **more** aggregate importance (2.8% vs 1.9%) — the
   categorisation redistributes information rather than discarding it.
3. Within the block, importance concentrates in the **populated** bins (B0, L0, C0).
   `C4` is exactly zero (39 jets) and B1–B4 are ~0 (empty) —
   4–5 of the 11 inputs are dead weight.

Summary of measured effects

change	full r_{95}	vs baseline
baseline (negrw, no SF)	**1343**	—
+ official 2D c-tag SF	**1371**	+2.1%
+ SF, with 2D-cat MVA scores	**1422**	+5.9%

- The SF costs **~2%** — the expected size and the correct sign for adding a real, previously-neglected uncertainty.
- Applying the SF **without** the matching 2D-cat discriminant is the mildest option, and is what `hww_combine_fixed` currently does.
- The matched (2D-cat + SF) combination is more *physically* consistent and genuinely separates better (stat-only −5%), but its wider SR admits more tt and loses on the full limit today.

Neither option is wrong — they trade discrimination against systematic exposure.
The choice should be revisited once the SF band is narrowed below `Total`.

Recommendations

1. Keep `up_Total` / `down_Total` as ONE nuisance for now. Full decorrelation (per-source, per-flavour, per-pt) is a whole-datacard exercise; the HiggsDNA `bTagShapeSF` treatment is the model to follow when it is done. **A `Total` band is conservative, not wrong.**

2. Collapse or drop the dead categories. B1–B4 receive **zero** candidate c-jets and C4 gets 39. Feeding 4–5 constant-zero inputs to the MVA is pure noise surface. Either collapse them into a single "B" bin, or apply the scheme to a jet collection that spans the plane.

3. Try the additive variant before spending v32 GPU time. Keep `cvsl` / `cvsb` **and** the one-hot categories. The only v11 regression was vs higgsbkg (−0.020), exactly where fine gradient matters — the additive model should recover it while keeping the +0.004 overall gain.

4. Extend to the other campaigns. The 2023 preBPix/postBPix and 2022 preEE SF files are already wired in; only 2022postEE is populated in the combine tree today.

Summary

2D c-tag SFs are wired in, validated, and cost ~2% of the limit.

The 11-category scheme needs only the two stored PNet scores —
no reprocessing (verified to machine precision).

Only 7 of 11 categories are populated for the candidate c-jet;
4 MVA inputs are identically zero.

1343 → **1371** with the SF · → **1422** with the matched 2D-cat discriminant

Full write-up: `Projects/HToWW/2026-07-19-ctag2d-full-documentation.md`

· reference: `References/HToWW/2D-SFbc-calibration-AN-25-222.pdf`

Backup

Central SF matrix, numerically

2022postEE, p_T = 60 GeV, $|\eta|$ inclusive.

flavour	L0	C0	C1	C2	C3	C4	B0	B1	B2	B3	B4
udsg	0.953	1.122	1.193	0.300	1.062	1.000	0.822	1.000	1.000	1.000	1.000
c	1.149	1.000	0.885	1.091	0.781	0.749	1.148	1.000	1.000	1.000	1.000
b	0.730	1.450	1.232	1.167	1.274	1.142	1.042	1.064	0.938	0.953	0.921

Note on the L0 convention: for L0, `up_Total` sits *below* `central` (udsg 0.925 < 0.953; c 0.300 < 1.149). L0 is the **anti-tag** category, so an upward shift in tagging efficiency *lowers* the veto SF. The band must therefore be taken as $|\mathrm{up} - \mathrm{central}|$ and $|\mathrm{central} - \mathrm{down}|$, **not** assumed ordered.

Across the whole 2022postEE MC every event lands in L0 / C0–C4 / B0; B1–B4 receive zero events (verified in the applier dry-run), so those SFs are exact no-ops regardless of value.

File map

what	where
processor helper	`~/higgscharm/analysis/utils/ctag2d.py` (branch `NewWorkflows`)
SF applier	`b-hive/scripts/apply_ctag2d_sf.py`
SF files	`/eos/cms/store/group/phys_higgs/cmshgg/ingredients/{2022,2023}/2D_HF_Tagging/`
backfill script	`b-hive/scripts/append_ctag2d.py`
one-hot appender	`b-hive/scripts/append_onehot.py`
2D-cat re-scorer	`b-hive/scripts/rescore_2dcat.py`
2D-cat combine workflow	`~/higgscharm/analysis/workflows/hww_combine_2dcat.yaml`
2D-cat combine tree	`outputs/hww_combine_2dcat/2022postEE/`
no-SF closure workflow	`~/higgscharm/analysis/workflows/hww_combine_nosf.yaml`
no-SF closure tree	`outputs/hww_combine_nosf/2022postEE/` (symlinks to `.bak_pre_ctag2dsf`)
2dcats train config	`b-hive/config/HPlusCHToWW_2dcats.yml`
feature importance	`b-hive/scripts/feature_importance_2dcats.py`
plane plot	`/eos/user/c/cgupta/HToWW/plots/ctag2d_plane_bins.{png,pdf}`

Two batch gotchas (cost hours, worth keeping)

Both hit the 2D-cat MVA retrain on the H100 pool:

1. `request_memory` **must be set.** The 3000 MB default OOMs in `DatasetConstructorTask` (peaks ~4.2 GB reading the big parquets) and the job is **held**, not failed — so it looks like it's still queued.

2. But do not inflate it. 16 GB rounds to `RequestMemory = 18000`, which then matches **zero** H100 slots (`condor_q -analyze` → "0 slots match … did not match any machines' constraints"). Free H100 GPUs are scarce and only the large memory tiers qualify, so over-requesting silently blocks matching.

8000 MB is the sweet spot — safely above the 4.2 GB peak, matched in ~8 s.

3. Training reads `mva_labeled/`**, not the top-level parquets.** The first submit died with `ValueError: key "cjet_cand_ctag2d_L0" does not exist` because the appender had only been run on the top-level files. Check the columns exist in `mva_labeled/{train,test}` before submitting.